Ingeniería de Soluciones con Inteligencia Artificial

INFORME EVALUACIÓN PARCIAL N°2

MecaIA — Agente funcional de IA para la mantención de autos y motos

Integrantes: Carlos Lienan · Alexis Ríos

Noviembre 2026

Repositorio	https://github.com/carlsLN/MecaIA (código, diseño, evidencias y README)
Framework	LangGraph (grafo de estados) con Claude como modelo de lenguaje vía LangChain
Lenguaje	Python 3.10 o superior
Estado	Agente funcional, 61 pruebas automáticas aprobadas, modo demo sin costo

1. Resumen

MecaIA es un agente de inteligencia artificial que orienta a personas dueñas de un auto o una moto en la mantención de su vehículo. Ante una consulta como «¿qué aceite necesito para mi Kawasaki 400?», el agente revisa primero si hay un riesgo de seguridad, confirma el modelo exacto del vehículo, busca el dato en una base de conocimiento con fuentes y responde con el dato, las herramientas necesarias, alternativas si no se tienen, el paso a paso y recordatorios, como cambiar el filtro junto con el aceite.

El agente recuerda los vehículos y las mantenciones de cada usuario, calcula cuándo toca la próxima y, cuando el caso excede lo que se puede hacer en casa, recomienda acudir al mecánico de confianza. Responde además dudas frecuentes: cada cuánto se hace cada mantención, cuánto demora y cuánto rinde el estanque.

2. Arquitectura del agente

El agente está implementado como un grafo de estados en LangGraph. Cada mensaje recorre los nodos de la figura 1; las aristas condicionales deciden el camino según la seguridad, la intención detectada y la información disponible.

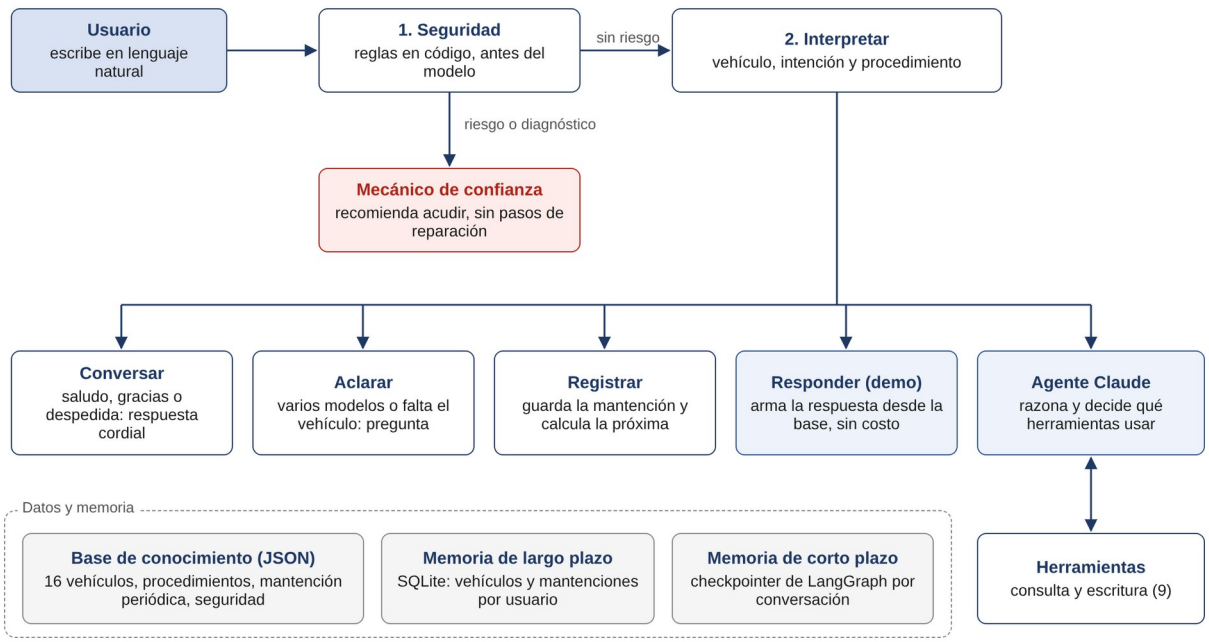


Figura 1. Flujo del agente MecaIA en LangGraph.

Nodo	Función
Seguridad	Detecta con reglas en código los riesgos (frenos que fallan, olor a combustible, humo) y los síntomas que requieren diagnóstico presencial (no enciende, ruido raro). En esos casos el turno termina recomendando acudir al mecánico de confianza, sin llamar al modelo.
Interpretar	Identifica el vehículo, la intención (procedimiento, intervalo, estanque, registro o conversación) y el procedimiento.
Aclarar	Pregunta cuando hay más de un modelo posible o falta el vehículo, y deja la pregunta pendiente en memoria.
Registrar	Guarda la mantención informada por el usuario y calcula la próxima.
Responder / Agente	En modo demo arma la respuesta desde la base. En modo Claude, el modelo razona y decide qué herramientas llamar.
Conversar	Responde saludos, agradecimientos y despedidas con cordialidad y ofrece seguir ayudando.

3. Herramientas de consulta, escritura y razonamiento

El modelo de lenguaje no responde desde su memoria: obtiene cada dato técnico mediante herramientas que leen la base de conocimiento o leen y escriben la memoria del usuario.

Herramienta	Tipo	Qué hace
buscar_vehiculo	Consulta	Encuentra los modelos que calzan con lo que dijo el usuario.
obtener_datos_vehiculo	Consulta	Aceite, capacidades, filtro, torques, estanque y rendimiento, cada dato con fuente.
buscar_procedimiento	Consulta	Pasos, herramientas, alternativas, recordatorios y cuándo acudir al mecánico.
consultar_mantenimiento_periodico	Consulta	Cada cuánto se hace una mantención y cuánto demora.
revisar_seguridad	Consulta	Evalúa el nivel de riesgo de lo que describe el usuario.
mis_vehiculos	Consulta	Vehículos guardados del usuario y su historial.
guardar_vehiculo	Escritura	Guarda o actualiza un vehículo en la memoria de largo plazo.
registrar_mantencion	Escritura	Registra una mantención realizada por el usuario.
recomendar_mecanico	Razonamiento	Define la urgencia y qué contarle al mecánico de confianza.

El razonamiento lo realiza Claude en el nodo Agente: interpreta preguntas libres, decide qué herramientas usar y en qué orden, y redacta la respuesta solo con los datos obtenidos.

4. Memoria de corto y largo plazo

Memoria	Implementación	Qué guarda
Corto plazo	Checkpointer de LangGraph (SqliteSaver), una conversación por identificador	Mensajes, vehículo confirmado y pregunta pendiente
Largo plazo	Base SQLite propia (mecaia/memoria.py)	Vehículos y mantenciones de cada usuario entre conversaciones



Figura 2. Uso combinado de la memoria de corto y largo plazo.

Gracias a la memoria de corto plazo, cuando el agente pregunta «¿cuál es tu Kawasaki?» y el usuario responde «la ninja», el agente retoma la consulta original. Gracias a la de largo plazo, días después el agente recuerda la moto y el último cambio de aceite sin que el usuario los repita.

5. Planificación y toma de decisiones adaptativas

El grafo fija el orden de las etapas: seguridad, interpretación, aclaración o respuesta. Dentro de ese plan, el agente adapta su camino según la situación:

Situación	Decisión del agente
El usuario describe un riesgo («mi moto no frena»)	No entrega pasos de reparación y recomienda acudir al mecánico de confianza
Síntoma que requiere diagnóstico («no enciende y hace un ruido raro»)	Explica que necesita revisión presencial y recomienda acudir al mecánico de confianza
Hay varios modelos posibles («Kawasaki 400»)	Pregunta cuál es, con opciones numeradas, antes de dar datos
El usuario no menciona el vehículo	Usa el vehículo de la conversación o el guardado en memoria
Falta un dato en la base	Lo declara y sugiere el manual, en vez de inventarlo
El usuario cuenta una mantención	La registra y calcula cuándo toca la próxima
El usuario agradece o se despide	Responde con cordialidad y ofrece seguir ayudando

Cada turno deja registrado su plan, que se muestra con la opción `--ver-plan`. Ejemplo real:

```
[plan] Seguridad: ok -> Vehículo: 3 candidatos -> Intención: consulta;
       procedimiento: cambio_aceite -> Decisión: preguntar antes de dar datos
```

6. Base de conocimiento y control de errores

La base de conocimiento está en archivos JSON versionados: 16 vehículos (5 autos y 11 motos de uso común en Chile), 3 procedimientos paso a paso, 10 ítems de mantención periódica y reglas de seguridad en dos niveles. Cada dato técnico lleva su fuente y un indicador `verificado`.

Para reducir las respuestas incorrectas («alucinaciones»), el agente aplica tres controles: los datos técnicos solo salen de la base; el chequeo de seguridad corre en código antes del modelo; y si un dato no existe, el agente lo dice. Los valores actuales son de referencia y están marcados como pendientes de verificar con los manuales oficiales.

7. Pruebas y evidencias

Grupo de pruebas	N°	Qué comprueba
Base de conocimiento	32	Identificación de los 16 vehículos y procedimientos; respuesta sin errores para cada vehículo

Grupo de pruebas	N°	Qué comprueba
Seguridad y diagnóstico	10	Detección de riesgos y de síntomas que requieren diagnóstico presencial; recomendación de acudir al mecánico de confianza, también en modo Claude
Intervalos y estanque	9	Cada cuánto, cuánto demora, autonomía, y que no se inventen datos faltantes
Memoria	4	Pregunta pendiente, vehículo entre conversaciones y registro de mantenciones
Conversación	4	Respuesta cordial a saludos, agradecimientos y despedidas
Herramientas y datos faltantes	2	Uso de herramientas en el bucle del agente (modo Claude) y que no se inventen datos de aceite

Las 61 pruebas se ejecutan con `python -m pytest -v`. El repositorio incluye el resultado (`evidencias/resultado_pruebas.txt`) y una conversación completa con el plan de cada turno (`evidencias/conversacion_demo.md`).

8. Ejecución, limitaciones y próximos pasos

```
pip install -r requirements.txt
python main.py --demo --ver-plan       # sin costo
python main.py --usuario ana          # con Claude (requiere ANTHROPIC_API_KEY)
```

- **Limitaciones:** los datos técnicos aún no están contrastados con manuales oficiales; el modo demo reconoce intenciones con palabras clave; la detección de riesgos usa reglas fijas.
- **Próximos pasos:** verificar los datos con una fuente experta, ampliar la cobertura de vehículos y procedimientos, y agregar una interfaz web.

Referencias

Anthropic. (2026). Claude API documentation. https://docs.claude.com

LangChain. (2026). LangGraph documentation. https://langchain-ai.github.io/langgraph/

Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.